

Reviewer Pack — Minimal Rank of Quadratic Reciprocity over Tseitin Resolution...

Entry 6d2c2dd0519a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 06:40:15 UTC

Minimal Rank of Quadratic Reciprocity over Tseitin Resolution Trees Width

Entry ID: 6d2c2dd0519a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 06:40:15 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Quadratic Reciprocity) **Field B** (complexity object): Complexity Theory: Tseitin Resolution Trees

Statement:

{‘p1’: ‘For any CNF formula F with n variables and m clauses, the width of the smallest Tseitin resolution tree refuting F is $\Theta(\sqrt{n} \log n)$.’, ‘p2’: ‘This implies that the minimal rank of the quadratic reciprocity matrix associated with F is also $\Theta(\sqrt{n} \log n)$, where the quadratic reciprocity matrix is constructed by evaluating the Legendre symbol of each possible pair of literals.’, ‘p3’: ‘Furthermore, for any prime p dividing the number of variables n , the determinant of the quadratic reciprocity matrix modulo p^2 is non-zero.’}

Rationale (proposer’s reasoning):

{‘p1’: ‘Quadratic reciprocity is a classical concept in number theory that relates to the Legendre symbol and the solvability of quadratic equations over finite fields.’, ‘p2’: ‘Tseitin resolution trees are used in complexity theory to analyze the proof size required for refuting CNF formulas, which is related to the difficulty of solving the satisfiability problem.’, ‘p3’: ‘Linking these two concepts could reveal a deeper connection between number-theoretic properties and computational complexity, potentially leading to new insights into the P vs NP problem.’}

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8d72b8395a8fd5a8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 90% of the constructed quadratic reciprocity matrices have a minimal rank within $\Theta(\sqrt{n} \log n)$, with all computed widths of Tseitin resolution trees refuting CNF formulas being $\Theta(\sqrt{n} \log n)$ and determinants modulo p^2 non-zero for all primes dividing n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_primes(n):
    primes = []
    num = 2
    while len(primes) < n:
        if is_prime(num):
            primes.append(num)
        num += 1
    return primes

def random_cnf(n, m):
    variables = list(range(1, n + 1))
    clauses = set()
    for _ in range(m):
        clause = []
        while len(clause) < 2:
            lit = random.choice(variables)
            if -lit not in clause:
                clause.append(lit)
```

```

        clause.append(lit)
        clauses.add(tuple(sorted(clause)))
    return clauses

def legendre_symbol(a, p):
    if a == 0:
        return 0
    if a % 2 == 1 and (p % 8 == 3 or p % 8 == 5):
        return -1
    if a % 2 == 1 and (p % 8 == 1 or p % 8 == 7):
        return 1
    if a % 2 == 0:
        e = 0
        while a % 2 == 0:
            a //= 2
            e += 1
        if e % 2 == 1 and (p % 8 == 3 or p % 8 == 5):
            return -1
        else:
            return 1

def quadratic_reciprocity_matrix(clauses, n):
    m = len(clauses)
    matrix = [[0] * m for _ in range(m)]
    for i in range(m):
        for j in range(i + 1, m):
            lit1, lit2 = clauses[i][0], clauses[j][0]
            if lit1 != -lit2 and lit2 != -lit1:
                matrix[i][j] = legendre_symbol(lit1 * lit2, n)
    return matrix

def tseitin_resolution_width(clauses):
    stack = []
    literals = set()
    for clause in clauses:
        literals.update(clause)
    for literal in literals:
        stack.append((literal, False))
    while stack:
        literal, negated = stack.pop()
        if negated:
            continue
        for clause in clauses:
            if literal in clause and not any(-x in clause for x in clause):
                for other_literal in clause:
                    if other_literal != literal:
                        stack.append((other_literal, True))
    return len(stack)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    m = 2 * n
    clauses = random_cnf(n, m)
    matrix = quadratic_reciprocity_matrix(clauses, n)
    width = tseitin_resolution_width(clauses)

```

```

min_rank = len(matrix)
for i in range(len(matrix)):
    for j in range(i + 1, len(matrix)):
        if matrix[i][j] != 0:
            min_rank -= 1
            break
    else:
        continue
    break

conjecture_holds = width == math.isclose(width, math.sqrt(n) * math.log(n), rel_tol=1e-5)
counterexample = "" if conjecture_holds else "mapping_undefined"
return {
    "metric_name": "width",
    "metric_value": width,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    std_width = math.sqrt(sum((r["metric_value"] - mean_width) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
    elif support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3f4a8c7.py", line 128, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3f4a8c7.py", line 102, in run_trial
    matrix = quadratic_reciprocity_matrix(clauses, n)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3f4a8c7.py", line 74, in quadratic_reciprocity_matrix

```

```
lit1, lit2 = clauses[i][0], clauses[j][0]
               ~~~~~^~^
```

TypeError: 'set' object is not subscriptable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13679
2	propose	ollama_remote	glm4:latest	0	0	10325
3	propose	ollama_remote	glm4:latest	0	0	11385
4	preregistration	ollama_remote	glm4:latest	0	0	6316
5	novelty	ollama_remote	glm4:latest	0	0	4755
6	novelty	ollama_remote	glm4:latest	0	0	5071
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	50306
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15924
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10055
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12984
11	judge	ollama_remote	glm4:latest	0	0	16780

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 157580 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/6d2c2dd0519a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6d2c2dd0519a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6d2c2dd0519a.tar.gz` (if generated)